

Autonomous exploration of indoor environments with a micro-aerial vehicle

Guillaume Pepe, Massimo Satler and Paolo Tripicchio

Abstract— The paper presents a system designed for micro-aerial-vehicles capable of autonomously explore an indoor environment, detect objects in the environment and build a map of the environment structure with reference to objects locations within the map. The found objects are saved in an internal database containing all previously recognized objects. The system allows fast exploration time and it is characterized by lightweight computation algorithms for the localization, map building and navigation components. The environment map is built as a set of 2D feature map layers where each layer corresponds to an environment floor. For each framework component a simulated testing scenario is presented to evaluate the capabilities of the designed algorithms.

The introduced system is efficient from the computational cost point of view and allows fast exploration time that is critical for battery powered systems.

I. INTRODUCTION

The exploration of an unknown area can be costly and unsafe for humans. Recently, there has been a growing interest in the employment of drones for inspection and exploration task thanks to the safety that this approach guarantees to the operator [1], [2]. Drones are nowadays employed for explorations, inspection of industrial or civil sites [3], [4], patrolling and so on. The employment of such technologies can be done according to different autonomy degrees: from teleoperated devices to completely autonomous systems. The choice is commonly related to the task to be accomplished and to the distance from the ground station (exploration of space for instance). Autonomous solutions are more challenging and complex to be implemented, on the other hand, teleoperated ones require skilled operators (people able to cope with the complexity of remote scene understanding, drone driving capability and even time-delayed teleoperation).

A recent trend consist in the employment of flying vehicles since they can access virtually any area and thanks to their agile dynamics and vertical take-off and landing capabilities [5], [6]. However, the employment of flying systems poses a number of challenges such as very limited payload capabilities, and hence limited computational resources.

G. Pepe is with Telecom Physique Strasbourg Engineering School of Strasbourg University, Strasbourg, France (guillaume.pepe90@gmail.com).

M. Satler is with Gustavo Stefanini advanced robotics research center of the TECiP Institute of the Scuola Superiore Sant'Anna, Pisa, Italy (m.satler@sssup.it).

P. Tripicchio is with Gustavo Stefanini advanced robotics research center of the TECiP Institute of the Scuola Superiore Sant'Anna, Pisa, Italy (p.tripicchio@sssup.it).

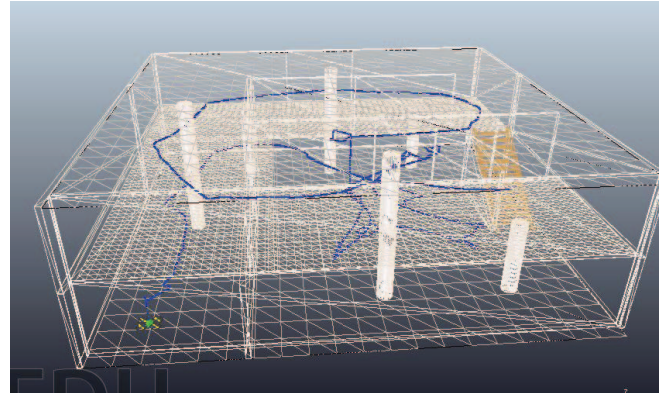


Figure 1. Multi floor indoor exploration with a MAV

Lot of research in this filed is related to algorithm design able to cope with localization in an unknown environment using on board sensor only [7], [8].

The work here presented is part of the research on autonomous exploration and object classification carrying out in several projects. In particular, the aim is to realize a system able to be employed in a MAV (Micro-Aerial-Vehicle) in order to autonomously explore a given environment and build in real-time the environment map and the database of the objects contained in the scene. The idea behind this work is to substitute the human being in hazardous and dangerous tasks.

In order to realize such a framework, it is required to cope with the following four main constraints:

- MAV localization using on-board sensor only;
- The MAV has to explore efficiently the environment taking into account the battery endurance;
- Obstacles avoidance module to prevent dangerous collisions;
- Objects detection and classification for database building.

The first fundamental issue to cope with is the localization of the vehicle in an unknown and unstructured environment. Moreover, given the limited payload and computational power, the localization algorithm will result in a trade-off between the required precision and the real-time constraint.

The desired system has to be able to work in outdoor as well as in indoor environment where the GPS is not reliable. A custom SLAM (Simultaneous Localization and Mapping) algorithm is used to compute the robot pose and some features in the environments knowing the sensors data, the control inputs and data association rules [7].

The term exploration implies the robot navigation through an unknown environment while building a map of the surroundings. Through the literature, it appears four main approaches to deal with this issue.

In [9] the frontier-based exploration is introduced, the frontiers represent the borders between the free already known space and the unknown environment. Those limits are detected thanks to an algorithm similar to an edges detector. In this approach, the robot moves towards the nearest frontier following the shortest path while avoiding obstacles given by the best-first search algorithm [10]. In [11], the robot performs a default policy exploration based on the Boustrophedon motion ([12], [13]) and revisits the interesting areas based on their visual saliency following a path defined again by a best-first search algorithm. In [14], virtual gas molecules are used to detect unknown areas, these particles move toward free and unknown space with a Brownian motion, colliding with already known obstacles. Once detected, the robot moves towards the navigation goals. In [15] a potential field is incrementally computed using harmonic functions where unknown areas have an attractive potential, whereas obstacles have a repulsive potential. In this way, the robot is attracted towards the nearest unexplored space. Our implemented exploration algorithm draws from this approach.

The remaining of the paper is organized as follow. Section II deals with the MAV localization based on laser scan and ultrasound measurements. The custom designed SLAM algorithm and the Kalman Filter for the altitude estimation are introduced and explained in detail. Section III presents the mapping algorithm employed to build the environment map. Section IV introduces the exploration algorithm based on the use of a virtual potential field. Section V shows the object detection and matching algorithm developed to recognize new objects in the scene using the RGB camera mounted on the vehicle. Conclusions and future works are reported in Section VI.

II. SLAM

In this section, we introduce the SLAM algorithm used to determine the robot pose on a 2D map (the Fast SLAM CGS) and an algorithm to obtain the drone altitude and levels elevation estimation using Kalman filters. Fusing the two algorithms it is possible to obtain a multi-layer map that describes a multi floor indoor environment.

A. 2D Pose

The Fast SLAM CGS [7] consists of eight main steps listed in Algorithm 1, each particle in the set $PARTICLES$ is described by its pose and its own map with N^k features. Each j^{th} feature of the k^{th} particle is represented with a mean $f_{j,t}^k$, a variance $F_{j,t}^k$ and a corresponding visibility counter: $i_{j,t}^k$ used to discard unreliable features.

In essence, the k^{th} particle is described as follow:

$$x_{t-1}^k; \left\{ \left(f_{1,t-1}^k, F_{1,t-1}^k, i_{1,t-1}^k \right) ; \dots ; \left(f_{N_{t-1}^k,t-1}^k, F_{N_{t-1}^k,t-1}^k, i_{N_{t-1}^k,t-1}^k \right) \right\}$$

FastSLAMCGS(PARTICLES)

1. **for** each *particle* in *PARTICLES*
do
2. Predict pose
3. Assign covariance pose
4. Find data association
5. Update pose
6. Sample the particle pose
7. Update particle features
8. Remove dubious features
9. **end for**
10. Resample(*PARTICLES*)
11. **Return** *PARTICLES*

Algorithm 1: SLAMCGS Overview

1) Aerial Vehicle Model

To test the algorithms presented in this paper The V-REP [16] simulation software has been used. A simple dynamic model of a quadrotor has been implemented for the exploration tasks presented in the following sections. The dynamic model is the one proposed in [17].

The quadrotor has been equipped with a simulated laser range finder (like an Hokuyo URG sensor), a virtual camera for image acquisition, an IMU complete with accelerometer, gyroscope and magnetometer and two sonars, one mounted on top of the quadrotor hull and pointing upwards and one mounted on the bottom of the quadrotor hull and pointing downwards.

2) Pose Prediction

To predict the drone pose, we first use the odometry data available from an embedded IMU. For the k^{th} particle, we have:

$$\widehat{\mu}_t^k = \begin{bmatrix} x_t^k \\ y_t^k \\ \psi_t^k \end{bmatrix} = g(x_{t-1}^k, u_t) = \begin{bmatrix} x_{t-1}^k + v_{x_{t-1}}^k dt + \frac{a_x dt^2}{2} \\ y_{t-1}^k + v_{y_{t-1}}^k dt + \frac{a_y dt^2}{2} \\ \psi dt \end{bmatrix}$$

With $\dot{\psi}$ the yaw rate given by the gyro sensor, a_x and a_y horizontal acceleration obtained from the accelerometer and the following transformation [18]:

$$a_w = \begin{bmatrix} a_x \\ a_y \\ a_z \end{bmatrix} = R_{IMU}^w a_{IMU} + \begin{bmatrix} 0 \\ 0 \\ -g \end{bmatrix}$$

$$R_{IMU}^w = \begin{bmatrix} c\theta c\psi & s\phi s\theta c\psi - c\phi s\psi & c\phi s\theta c\psi + s\phi s\psi \\ c\theta s\psi & s\phi s\theta s\psi + c\phi c\psi & c\phi s\theta s\psi - s\phi c\psi \\ -s\theta & s\phi c\theta & c\phi c\theta \end{bmatrix}$$

With g the gravity acceleration constant, $c=\cos$, $s=\sin$, ϕ, θ, ψ roll, pitch and yaw angle obtained from the gyro sensor.

Then, the robot pose covariance is set equal to the white noise δ_t covariance matrix R_t .

3) Landmark extraction

It appears the walls are the most obvious landmarks for an indoor environment according to two main reasons. First, they

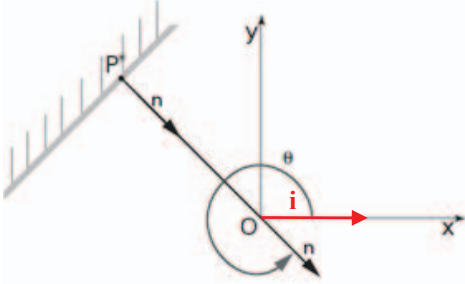


Figure 2. Hessian representation of a wall

are commonly present and then their extraction is robust and reliable. We used the Hessian representation [19] to easily model the walls coordinates.

Using the Hessian representation, the wall coordinate are given by the triplet: (r, θ, v)

- $r = ||\mathbf{OP}^*||$ the distance from O to the closest point P^* in the wall
- θ represents the angle **counter-clockwise** between the vector $\mathbf{i}$ and the outward normal $\mathbf{n}$ of the wall. θ belongs to the interval $[0, 2\pi)$ and is computed using the function acos2 defined below:

$$\theta = \text{acos2}(\mathbf{i}, \mathbf{n}) = \begin{cases} 2\pi - \text{acos}(\mathbf{i}, \mathbf{n}), & \text{if } \det(\mathbf{i}, \mathbf{n}) < 0 \\ \text{acos}(\mathbf{i}, \mathbf{n}), & \text{if } \det(\mathbf{i}, \mathbf{n}) \geq 0 \end{cases}$$
- $v = -\frac{\mathbf{OP}^* \cdot \mathbf{n}}{|\mathbf{OP}|}$ states if the frame O is front of the wall $v = 1$ or behind it $v = -1$

Landmarks are extracted from the laser range finder data through a RANSAC (*Random Sample Consensus*) line extraction algorithm [20].

Firstly, in order to easily compute the RANSAC algorithm we transform laser data into Cartesian coordinates such as:

$$\begin{bmatrix} x_{\text{point}} \\ y_{\text{point}} \end{bmatrix} = \begin{bmatrix} \text{range}_{\text{laser}} \cdot \cos(\text{bearing}_{\text{laser}}) \\ \text{range}_{\text{laser}} \cdot \sin(\text{bearing}_{\text{laser}}) \end{bmatrix}$$

After taking 2 points, we use the least squares method to fit a line between those points, thus obtaining the parameters a and b of the Cartesian form (1) using equations (2) and (3).

$$y = ax + b \quad (1)$$

$$a = \frac{N \sum_{i=1}^N x_i y_i - \sum_{i=1}^N x_i \sum_{i=1}^N y_i}{N \sum_{i=1}^N x_i^2 - (\sum_{i=1}^N x_i)^2} \quad (2)$$

$$b = \frac{1}{N} (\sum_{i=1}^N y_i - a \sum_{i=1}^N x_i) \quad (3)$$

With N the number of data points.

We compute the distance from each point $p(x, y)$ to the line using the following formula:

$$d = \frac{|ax - y + b|}{\sqrt{a^2 + b^2}}$$

Once obtaining the inliers set I with a sufficient number of points, we fit a new line and extract the parameters of the polar form (4), thanks to the algorithm *ExtractPolarForm* [21].

$$x \cos(\alpha) + y \sin(\alpha) = \rho; \rho \geq 0 \quad (4)$$

Finally, the covariance matrix of the line parameters: $Q =$

$$\begin{bmatrix} \sigma_\rho^2 & \sigma_{\rho\alpha} \\ \sigma_{\rho\alpha} & \sigma_\alpha^2 \end{bmatrix}$$
 is computed according to [21].

The lines extraction algorithm has been implemented on Matlab using laser data simulated on V-REP. The laser data set is about 1080 points.

4) Measurement model

To predict the laser rangefinder measurement, we use the prediction target response for planes defined in [22].

We consider the feature f_n modelled by the hessian triplet (r_n, α_n, v_n) and the sensor pose

$$\mathbf{x} = (x_t \ y_t \ \psi_t)^T.$$

The measurement prediction $\hat{z}$ is computed as follow:

$$\hat{z} = \begin{bmatrix} \hat{r} \\ \hat{\alpha} \end{bmatrix} = h(f_n, x_t) = \begin{bmatrix} v_n(r_n - x_t \cos(\alpha_n) - y_t \sin(\alpha_n)) \\ \alpha_n - \psi_t \end{bmatrix}$$

We obtain $H_F = \nabla_F h(f_n, x_t)$, the Jacobian of the measurement prediction with respect to the landmarks:

$$H_F = \begin{bmatrix} v_n & v_n(x_t \sin(\alpha_n) - y_t \cos(\alpha_n)) \\ 0 & 1 \end{bmatrix}$$

The Jacobian H_x of h with respect to the pose is computed as follows:

$$H_x = \nabla_x h(f_n, x_t) = \begin{bmatrix} -v_n \cos(\alpha_n) & -v_n \sin(\alpha_n) & 0 \\ 0 & 0 & -1 \end{bmatrix}$$

The correspondences between the detected features and the already known landmarks are found using the maximum likelihood (line 4 of the SLAM CGS overview). If this value is under a certain threshold p_0 , the current detected feature is considered as a new landmark.

Before the importance resampling step, as mentioned before, the unreliable particles features are removed thanks to a visibility counter. Indeed, if a measure is not associated, a feature in the particle map is considered as a new landmark. Therefore its visibility counter is initialized as $i_j^k = 2$.

Then, detecting a feature confirms its existence, so every time a particle feature is associated to a detected landmark, its counter is incremented $i_j^k = i_j^k + 2$. While with each cycle, every visible particle feature (localized in the laser field), has its counter decremented. Finally, particles features with a visibility counter under a certain threshold are removed.

The basic importance resampling is achieved using the *Low Variance Sampler* [23] method.

5) Test

The Fast SLAM CGS algorithm has been implemented on Matlab, the experiment was carried out in a simulated indoor environment in V-REP. The robot trajectory consists of 676 points with a 20 Hz sampling frequency.

The number of particles is set equal to **100** and for each one the maximum number of features is limited to **12**. The new feature p_0 threshold is set equal to **0.1**.

Figure 3 depicts the actual drone trajectory (black line), the estimated one by the odometry data (red line) and by the SLAM CGS (blue x). The mean error between the actual and estimated trajectory is about 8 cm for the x and y axis.

Figure 4 shows the actual and estimated drone orientation.

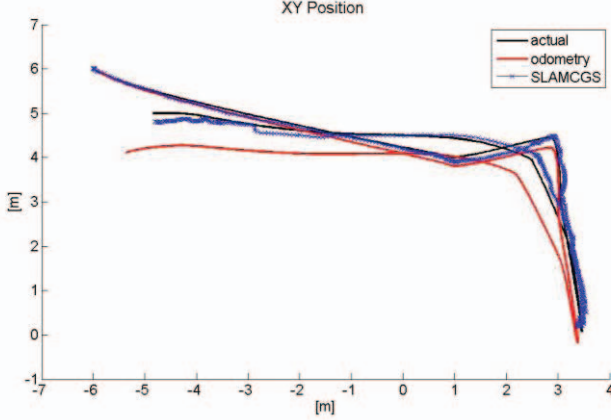


Figure 3. Trajectory performed by the MAV

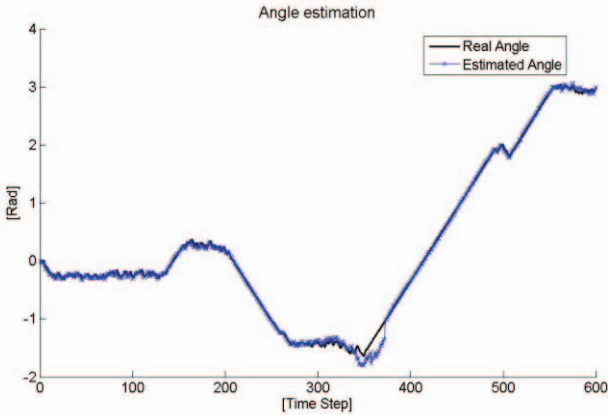


Figure 4. Actual and estimated drone angle

B. Altitude and Levels

Since the barometric sensor is potentially unworkable in industrial environment because of the presence of a temperature gradient, the drone altitude can't be extracted from this device.

In addition, the drone can fly over a non-flat ground and obstacles. Therefore, we cannot extract a global altitude for the drone only using a downwards sonar and a Kalman filter. For this reason, the drone height is computed from the upwards and downwards sonars data using an altitude estimation algorithm consisting of Kalman filters.

S.Gronzka has developed the Multilevel SLAM algorithm ([24] and [25]) to track the drone height and the ground levels. In addition, we slightly modified this algorithm in order to take into account the upwards sonar data and therefore estimate the ceiling levels.

Basically, the algorithm is computed in two main step. First, the drone altitude is updated from the sensors data and the already known levels. Then these different levels are updated using individual Kalman filters. It's assumed the drone pose (x_t, y_t, ψ_t) is already known from the Fast SLAM CGS.

The very first step of the algorithm is to compute the drone height and vertical velocity according to the model prediction, as detailed below:

$$\hat{x}_t = \begin{bmatrix} \hat{z}_t \\ \hat{v}_{z,t} \end{bmatrix} = A \begin{bmatrix} z_{t-1} \\ v_{z,t-1} \end{bmatrix} + B a_z$$

$$\text{with } A = \begin{bmatrix} 1 & dt \\ 0 & 1 \end{bmatrix}; \quad B = \begin{bmatrix} 0.5dt^2 \\ dt \end{bmatrix}$$

$$\hat{\Sigma}_t = A \Sigma_{t-1} A^T + R \quad \text{with } R = \begin{bmatrix} \sigma_z^2 & \sigma_z v_z \\ \sigma_z v_z & \sigma_{v_z}^2 \end{bmatrix}$$

Then, we predict the ground and ceiling elevation beneath and above the drone according to the measurement prediction:

$$\hat{h}_t = \begin{bmatrix} \hat{h}_{ground} \\ \hat{h}_{ceiling} \end{bmatrix} = \begin{bmatrix} \hat{z}_t - z_{downwards} \\ \hat{z}_t + z_{upwards} \end{bmatrix}$$

The next step is to find the matches with the already known levels close to the robot current pose.

Once obtaining the matches, those are merged into a single level elevation. In [25], two levels are merged into one measure according to the following formula:

$$(\mu_{12}, \sigma_{12}^2)^T = \left(\frac{\sigma_1^2 \mu_2 + \sigma_2^2 \mu_1}{\sigma_1^2 + \sigma_2^2}, \frac{\sigma_1^2 \sigma_2^2}{\sigma_1^2 + \sigma_2^2} \right)^T$$

To merge N levels from a set C, one can prove that it can be achieved using the next equation:

$$(\mu_{1:N}, \sigma_{1:N}^2)^T = \text{mergeLevels}(C) = \left(\frac{\sum_{k=1}^N \mu_k \prod_{j=1, j \neq k}^N \sigma_j^2}{\sum_{k=1}^N \prod_{j=1, j \neq k}^N \sigma_j^2}, \frac{\prod_{k=1}^N \sigma_k^2}{\sum_{k=1}^N \prod_{j=1, j \neq k}^N \sigma_j^2} \right)^T \quad (5)$$

After merging the levels elevations, the drone altitude and the current levels elevations beneath and above the drone are updated as follows.

$$Q_t = \sigma_{laser} + \sigma_{ground}$$

$$K = \hat{\Sigma}_t D^T (D \hat{\Sigma}_t D^T + Q)^{-1} \quad \text{with } D = [1 \ 0]$$

$$x_t = \begin{bmatrix} z_t \\ v_{z,t} \end{bmatrix} = \hat{x}_t + K(\mu_{ground} + z_{downwards} - D \hat{x}_t) \quad (6)$$

$$\Sigma_t = (I_2 - KD) \hat{\Sigma}_t$$

$$\hat{h}_t = \begin{bmatrix} \hat{h}_{ground} \\ \hat{h}_{ceiling} \end{bmatrix} = \begin{bmatrix} z_t - z_{downwards} \\ z_t + z_{upwards} \end{bmatrix}$$

Updating the drone altitude and levels from the ceiling measurement, implies to modify the equation (6) such as:

$$x_t = \hat{x}_t + K(\mu_{ceiling} - z_{upwards} - D \hat{x}_t)$$

If no matches have been found between the estimated levels and the already known elevation ($C_{ground} = \emptyset$ and $C_{ceiling} = \emptyset$), the best estimation of the drone altitude has already been provided by the prediction step.

Once updating the drone altitude, the levels elevation are updated using for each one a Kalman filter. Then, the new levels are inserted in the map M.

Finally, the last step of the algorithm aims to merge closest levels. We only consider the levels close to the current drone

pose. Afterward, within this subset, we compare the difference between the levels elevation, if it's under a certain threshold δ_2 , we merge them according to the equation (5). The map is therefore updated.

1) test

The Multilevel SLAM algorithm has been implemented on Matlab and tested with data from V-REP. The drone embeds two sonars (upwards and downwards) and an accelerometer, the sampling frequency is set equal to 20 Hz. As shown in the Figure 7, the drone flies above a 0 m high ground, a 0.55 m high table, the first floor was 2.41 m above the ground, the ceiling 4.66 m above it, the scenario is depicted in Figure 5.

The map M is initialized with a 0.0 m high ground and 15.0 m high ceiling to avoid drifting.

After performing some tests, the variance σ_m of levels altitude is set equal to **0.10 m**, the range R around the robot to consider its neighbors is set to **1 m**, the thresholds for levels merging δ_1 and δ_2 are respectively set to **0.20 m** and **0.05 m**. The estimated altitude is shown in Figure 6.

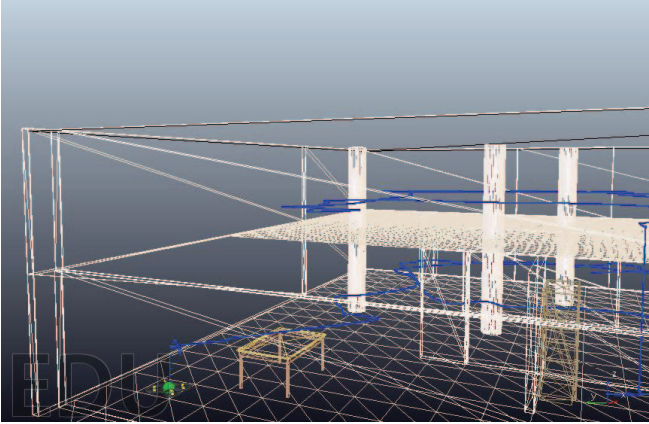


Figure 5. V-REP scenario for the Multilevel SLAM test

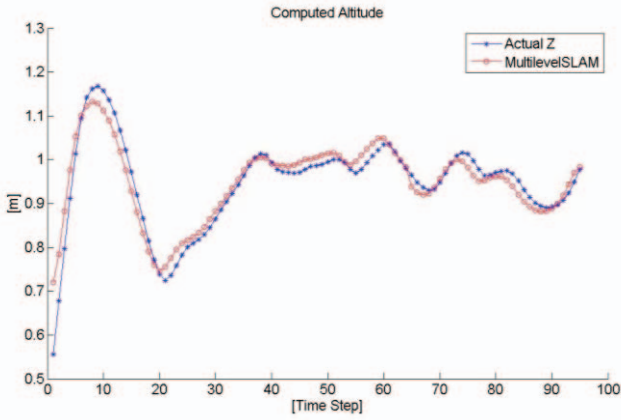


Figure 6. Estimated Altitude

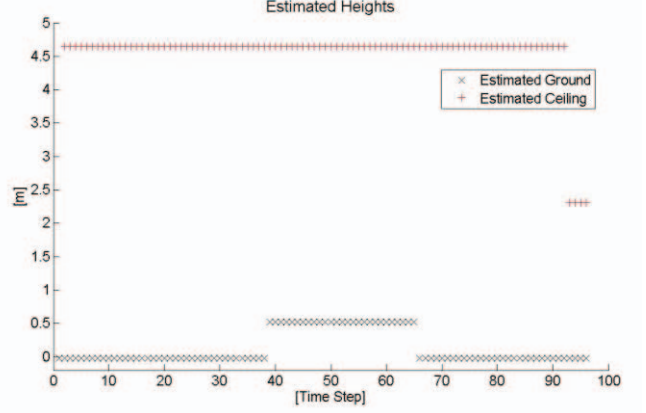


Figure 7. Detected heights levels

The mean of the error between the actual and estimated drone altitude is about **2.3 cm** for the first test and **1.8 cm** for the second one, this fits to the value given in [24].

Levels	Actual height	Estimated test1	Estimated test2
Ground	0.0 cm	-1.85 cm	0.8 cm
Table	55.0 cm	52.1 cm	48.9 cm
Ceiling	4.66 m	4.65 m	4.67 m
First floor	2.41 m	2.31 m	2.33 m

III. MAPPING

At this point the drone pose is estimated from the two localization algorithms, however the only knowledge of the walls position is not sufficient to allow the autonomous drone navigation inside an unknown indoor environment.

Therefore, the next step to explore the environment is to create a map of it. Indeed, in order to plan a path ensuring obstacles avoidance within the robot surroundings, one needs to estimate an accurate map from the laser range scanner data. Our approach to build the indoor map is based on the occupancy grid mapping [23].

The basic idea of the occupancy grid algorithm is to compute the posterior probability of the map only knowing the observation data and the robot path previously estimated such as:

$$p(m|z_{1:t}, x_{1:t}) \quad (7)$$

The space is partitioned into a small grid cells where each cell m_k has an occupancy probability $p(m_k)$ which takes a value between 0 (free) and 1 (occupied).

Nevertheless, the equation (7) can't be directly computed due to the map size, therefore the map posterior is approximated by computing the posterior for each discrete cell within the partitioned space M .

$p(m|z_{1:t}, x_{1:t}) = p(m_k|z_{1:t}, x_{1:t})$, for each cell $m_k \in M$
Because the state of the cell is necessarily static: occupied or free, it has been proved in [23] that its occupancy probability can be computed recursively using Bayes rules as follows:

$$l_{k,t} = l_{k,t-1} + \log \frac{p(m_k|z_t, x_t)}{1-p(m_k|z_t, x_t)} - \log \frac{p(m_k)}{1-p(m_k)}$$

With $l_{k,t} = \frac{\log p(m_k|Z_{1:t}, x_{1:t})}{1-p(m_k|Z_{1:t}, x_{1:t})}$, the log odds ratio form of occupancy probability, used to avoid instabilities. The cell occupancy probability is then extracted through the formula:

$$p(m_k|Z_{1:k}, x_{1:k}) = 1 - \frac{1}{1+\exp(l_{k,t})}$$

1) Test

The Occupancy Grid Mapping algorithm has been implemented on Matlab and tested with data from V-REP. The scene is an environment cube consisting of two floors connected by a staircase, 5 and 3 pillars are respectively located in the first and second floor. The drone explores the two floors.

The occupancy grid is 16x16 m large with a 10x10 cm resolution, each grid has initially its probability occupancy set equal to 0.5 (unknown state). The obstacles thickness and the angular width beam are respectively set equal to **0.5 m** and **$8.8 \cdot 10^{-3}$ rad**.

The extracted maps (Figure 8) fairly enough represents the robot environment and can be used for path planning and obstacles avoidance. Nevertheless, increasing the grid resolution will improve the map accuracy but slow down the algorithm.

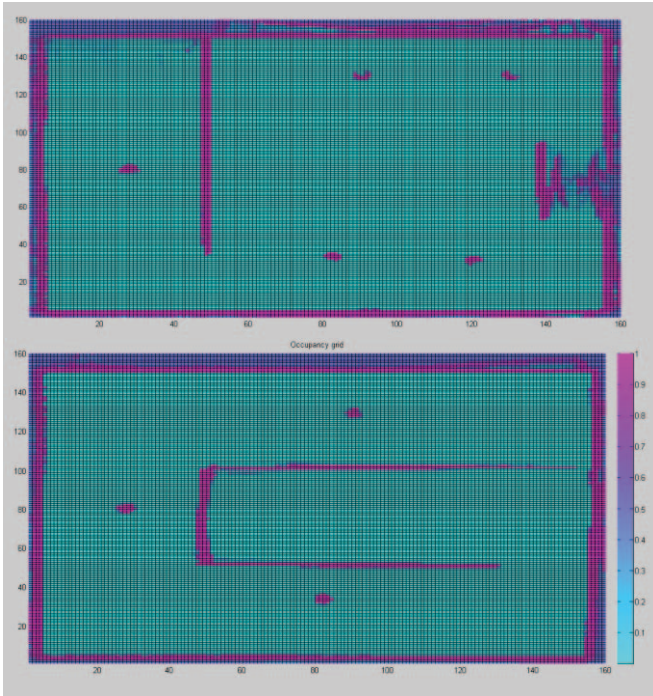


Figure 8. Grid maps of the simulated environment

IV. EXPLORATION

We chose the approach presented in [15] for our exploration policy for its efficiency, easy implementation and above all because it avoids the use of best-first algorithm which can be time-consuming if the goal is far from the robot.

The potential field based path planning was introduced by Khatib [26]. Commonly, the navigation goal has an attractive potential and the obstacles a repulsive one, by

summing the attractive and the repulsive forces undergone by the robot, the optimal path leading to the goal while avoiding obstacles follows the negative gradient of the virtual potential.

Few years later, Connolly and Grupen [27] solved the issue of the local minimums for potential field based navigation, using a new potential field computed from harmonic functions.

In [15], the exploration is performed in 3 main steps, first a map is built using Histogramic In-Motion Mapping (HIMM) [28]. Then, the potential is updated within a sliding window, this means that cells which belong to the circle of a certain radius R centered in the robot pose have their potential updated according to the Gauss-Seidel method. Cells with a high occupancy probability have their potential set equal to 1. At the begin of the exploration all cells have their potential set equal to 0, therefore unknown areas are cells which have never been updated inside the sliding window, they represent navigation goals and the robot is attracted to them. The final step is to compute the gradient on the current robot cell and move following the gradient descent.

In our approach, we use the algorithm previously described but we use the built map from the occupancy grid mapping instead of the HIMM method.

A. Single Floor

The first step of our exploration algorithm is to update the potential field inside the sliding window. As suggested in [29], the potential value of the cell containing the robot is set equal to 1. Later, cell with an occupancy probability above a certain threshold are considered as obstacles and therefore their potential is set equal to 1. The potential of the free cells is updated according the Gauss-Seidel method while taking into account the cells in the borders.

Once the desired accuracy is obtained, the negative gradient is numerically computed. Finally, to follow the gradient descent, we use the method described in [30]. A reference pose is computed at each time step in order to follow the optimal path towards the goal position (unknown space).

1) Test

The harmonic potential field based exploration algorithm has been implemented and tested on Matlab. For the tests, we have created two 15x15m mazes, the resolution of the potential map is about **10 cm**. The radius of the sliding window is set equal to **2 m**, the maximum relative error is set to **$1e^{-4}$** as suggested in [29].

Figure 9 shows the environment for the first exploration test, the dark areas represent obstacles, the initial robot pose is $(-6, 6, 0)$ (red circle in the figure).

The exploration ends when the percentage of cells with an unknown state namely with a potential value equal to 0 is less than **5%**. In the figure the robot path during the exploration is shown.

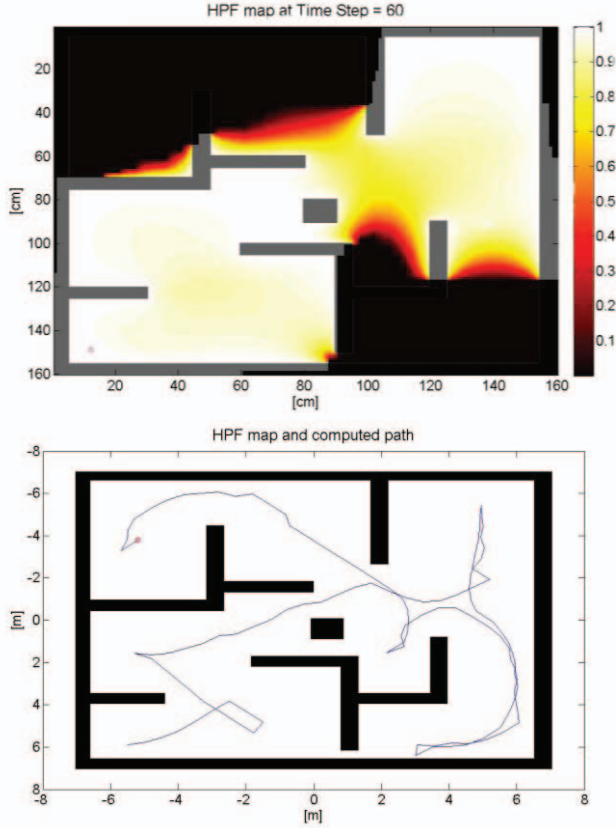


Figure 9. Single Floor exploration Scenario

However, like with the experiments in [15] the robot tends to oscillate when obstacles are numerous and quite close, therefore the robot spends time following a not optimal path during the exploration.

B. Multi-Floor

At this point, the drone is able to only explore a single floor indoor environment. Hence, we would like to extend the exploration for multi-floor environments combining the detected levels with the Multilevel SLAM algorithm and the harmonic potential field based exploration.

Once performing the current floor exploration, the next step is to determine if a higher floor exists, this is done by extracting modes from the Multilevel SLAM data. Indeed, if a top floor exists the distribution of the upwards sonar data should be bimodal. The figure below (Figure 10) shows an example of this situation.

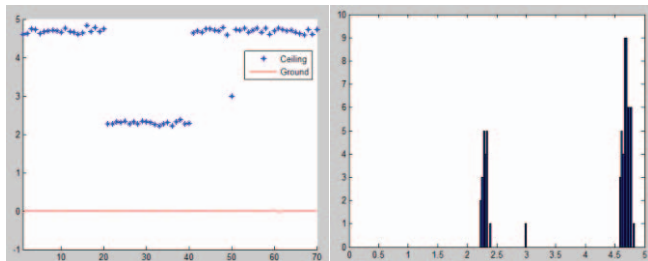


Figure 10. Detected levels and modes

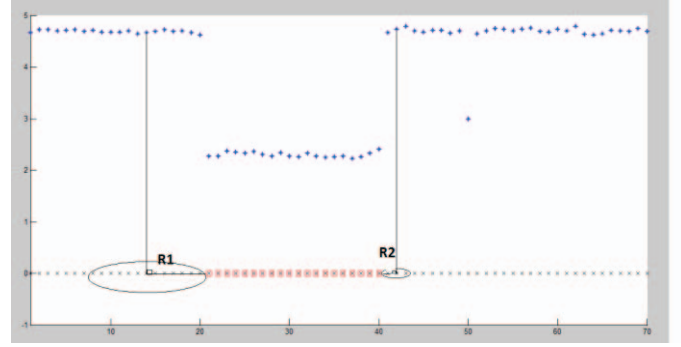


Figure 11. Transition points recruitment

The 2 modes of the distributions are extracted using the k-means method which aims to split the data into k clusters. In this case, we want to extract 2 clusters and their centroid. This provides us the ceiling and upper floor height.

Then, if the difference between the ceiling elevation and the upper floor height is above a certain threshold, we find the transition points namely the points through which the robot can safely go to the next level. From the Multilevel SLAM algorithm, each detected level has its own height and Cartesian coordinates, therefore we select points which are on the ceiling height and we check where their closest “downstairs neighbors” are. If the distance R to the nearest neighbor is above a safety distance this point can be considered as a transition point. Figure 11 depicts the transition points selection, the point on the left has its closest “downstairs neighbor” at a distance equal to $R1$, for the right point this distance is equal to $R2$.

Finally the selected transition point is chosen as the nearest point among the transition candidates previously found.

Then the robot reaches this transition point following a path given by the harmonic potential field based exploration algorithm. Indeed, setting the potential of the cell containing the goal point equal to a very low value knowing the explored environment, the previous algorithm becomes a goal oriented path planner to reach the navigation goal.

Once reaching this point, the drone goes to the upper floor. The exploration ends when there is no more detected upper floor, this means that the upwards sonar measurement distribution has only one mode. Or it could also happens when the height difference between the ceiling and the upper floor is too small. Algorithm 2 sums up the Multi-floor exploration approach.

1) Test

The Multi-floor exploration algorithm has been implemented on Matlab and tested with V-REP. The V-REP scene is a $16 \times 16 \times 4.8 \text{ m}^3$ closed environment with two floors connected by a staircase. The sliding window radius for the harmonic potential field based exploration is set equal to **3 m**, the minimum height difference threshold between the upper floor and the ceiling is set equal to **1.5m** and the maximum unknown area percentage set equal to **10%**. The safety distance between a transition point and its nearest “downstairs neighbors” is also set equal to **1.5 m**, this


```

MultiFloor_Exploration( $X, M, \Phi$ )
EXPLORE=TRUE
while EXPLORE do
  A. Explore the current floor using HPF_Exploration() until the size of the unknown area is under
  a certain threshold
  B. From the MultiLevelSLAM() data determine if a upper floor UF exists using the k-means
  method
  if  $\exists UF$  do EXPLORE=FALSE end if
  C. Find the transition point TP
  if  $\exists TP$  do EXPLORE=FALSE end if
  else Compute the path using HPF_Exploration() to the goal navigation from the current
  robot pose
  end if
  D. Move to the transition point following the previous path
  E. Go to the upper floor
end while

```

Algorithm 2: MultiFloor_Exploration algorithm

represents the double of the drone radius. During the exploration, the drone has to fly **1 m** over the current ground. We use a *k-means* detector to obtain the upwards sonar distribution modes.

The drone trajectory during the 11 minutes exploration is shown by the blue line. The computation of the path to reach the transition point is about 1 minute on Matlab, given the necessity to update the potential for the whole grid.

It can be observed on the Figure 12 that the drone explored large areas while avoiding to inspect narrow corridors. This can be changed by decreasing the maximum unknown area percentage.

V. VISUAL DATABASE

During the exploration, by using its front RGB camera the drone is able to detect object on the ground. Then, every new detected object is inserted in database containing images from different poses and reference to its location on the map. To do so, we extract objects inside the current image thanks to a color based segmentation algorithm. Then by using the SURF (Speed Up Robust Features) algorithm [31] which finds matches between two images, it is possible to determine if the detected object has been already met, if it is not, the current object is inserted in the database, otherwise only the map reference is updated .

A. Color Based Segmentation

The color segmentation to extract an object from a scene is based on thresholds. Indeed, for each RGB component, we set the pixels with an intensity value under a certain threshold I_0 equal to 0, otherwise it is set equal to 1 as follows:

$$\begin{cases} I_{segmented}^C(i, j) = 0, & I(i, j) < I_0 \\ I_{segmented}^C(i, j) = 1, & I(i, j) \geq I_0 \end{cases}$$

with (i, j) the pixel coordinates. Then the segmented image is provided from the three segmented images.

$$I_{segmented} = I_{segmented}^R + I_{segmented}^G + I_{segmented}^B$$



Figure 12. Multi-Floor environment exploration

Next, the noise is removed by using a morphological filter. Finally, we extract with the Matlab function *regionprops()* the centroid and size of each detected object.

B. Objects recognition

In order to recognize an object, we have to find matches between two images: the object reference image and the current scene. This is done in three main steps. First, *interest points* are extracted from the image for their distinctiveness (ex: Harris corner, circle...). Then, each interest point neighborhood is described by a vector, which is called

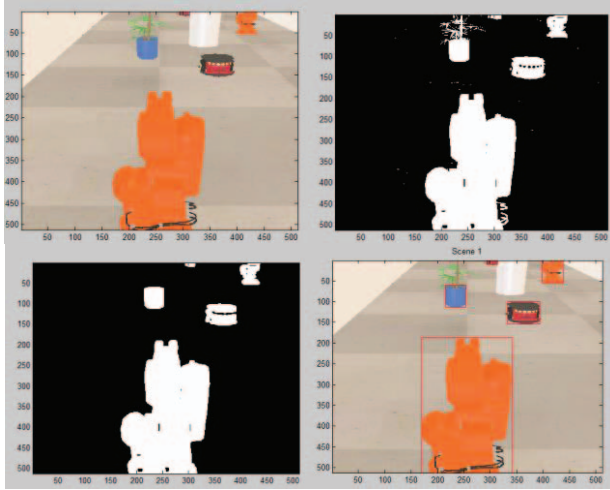


Figure 13. Object Segmentation and detection

feature descriptor (ex: histogram of local oriented gradients [32]). Finally, the correspondences are found by computing the distance between the feature vectors.

We use the SURF algorithm for its light computation time and robustness to scale and orientation transformations which allows us to recognize an object from different camera point of view. In a word, it uses the determinant of the Hessian matrix as interest points detector and the Haar-wavelets as feature descriptors.

It's worth noting that the objects recognition is limited to objects with texture but can be extended to use also geometrical features.

VI. CONCLUSION AND FUTURE WORK

Through this study, we searched a full approach to solve the autonomous exploration of an unknown indoor environment using a micro-aerial vehicle with a short autonomy only embedding sonars, IMU, a 2D laser range finder and a camera. This method consists of four main steps.

Firstly, the robot pose and its surroundings are obtained with a reasonable error from the sensors data with two lightweight computation algorithms namely the SLAM CGS and the Multilevel SLAM using a particle filter and low dimensional Kalman filters. During the testing sessions the pose estimation has been limited to indoor environments with walls but could be extended using different landmark features.

The second step completes the robot environment knowledge through the use of the occupancy grid mapping. In our study, we limited each floor to a 2D map with a regular grid thus reducing the amount of information for the whole indoor environment. Then, the exploration policy is carried out using a virtual potential field with harmonic functions which avoid to trap the robot into local minima. The experiments show good results for 2D cases and fits to the indoor. Nevertheless, this approach implies to roughly know the size of the environment. The extension of the 3D case works efficiently with multi-floors structures such as offices. A complete 3D representation could conversely be

obtained using a more sophisticated approach like quad-trees for the mapping and extensive 3D scans of the environment with the drawback of increased computation needs and increased exploration time.

For the last part, the detection and recognition of objects is performed by a simple color based segmentation method and the SURF algorithm. This approach shows limitations with texture-less objects. We plan to extend the recognition phase as future work reconstructing the detected objects in 3 dimensions and finding a control law to drive the aerial vehicle in order to get enough images for this reconstruction.

REFERENCES

- [1] LI, Yanming, et al. "A novel mobile robot for finned tubes inspection", *Robotica*, 2003, 21.06: 691-695.
- [2] Nagatani, Keiji, et al. "Volcanic ash observation in active volcano areas using teleoperated mobile robots-Introduction to our robotic-volcano-observation project and field experiments", *Safety, Security, and Rescue Robotics (SSRR)*, 2013 *IEEE International Symposium on*. IEEE, 2013.
- [3] M.Satler, M.Unetti, N.Giordani, C.Avizzano and P.Tripicchio, "Towards an autonomous flying robot for inspections in open and constrained spaces", in *The 11th IEEE International Conference on Sensors, Circuits and Instrumentation Systems*, Barcelona, 2014.
- [4] Nikolic, J., Burri, M., Rehder, J., Leutenegger, S., Huerzeler, C., & Siegwart, R. (2013, March). "A UAV system for inspection of industrial facilities", In *Aerospace Conference*, 2013 *IEEE* (pp. 1-8). IEEE.
- [5] Z. Sarris, "Survey of UAV Applications in Civil Markets", in *Mediterranean Conference on Control and Automation*, Ancona, Italy, 2001.
- [6] N. Metni and T. Hamel, "A UAV for bridge inspection: Visual servoing control law with orientation limits", *Automation in Construction*, vol. 17, no. 1, pp. 3–10, November 2007.
- [7] P. Tripicchio, M. Unetti, N. Giordani, C. Avizzano and M. Satler, "A Lightweight SLAM Algorithm for Indoor Autonomous Navigation", in *Australasian Conference on Robotics and Automation (ACRA)*, 2014.
- [8] M. Achtelik, M. Achtelik, S. Weiss, and R. Siegwart, "Onboard imu and monocular vision based control for MAVs in unknown in- and outdoor environments", in *Proc. of the IEEE International Conference on Robotics and Automation (ICRA)*, 2011.
- [9] B.Yamauchi, "A Frontier-Based Approach For Autonomus Exploration", in *Symposium on Computational Intelligence in Robotics and Automation, 1997. CIRA'97., Proceedings., 1997 IEEE International*, Monterey, CA, July 1997.
- [10] P.Hart, N.Nilsson and B.Raphael, "A formal basis for the heuristic determination of minimum cost paths", *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100-107, 1968.
- [11] A.Kim and R.Eustice, "Perception-driven navigation: Active visual SLAM for robotic area coverage", in *2013 IEEE International Conference on Robotics and Automation (ICRA)*, Karlsruhe, 2013.
- [12] H.Choset and P.Pignon, "Coverage path planning: the Boustrophedon decomposition", in *Proc. International Conference Field and Service Robotics Dec.*, 1997.
- [13] A.Ntawumenyikizaba, H. H. Viet and T.Chung, "An online complete coverage algorithm for cleaning robots based on boustrophedon motions and A* search", in *2012 8th International Conference on Information Science and Digital Content Technology (ICIDT)*, Jeju Island, South Korea, 2012.
- [14] S.Shen, N.Michael and V.Kumar, "Autonomous indoor 3D exploration with a micro-aerial vehicle", in *2012 IEEE International Conference on Robotics and Automation (ICRA)*, Saint Paul, MN, 2012.

- [15] E.Silva, P.Engel, M.Trevistan and M.Idiart, “*Exploration method using harmonic functions*”, Robotics and Automation Systems, vol. 40, no. 1, pp. 25-42, 2002.
- [16] E.Rohmer, S.Singh and M.Freese, “*V-rep: A versatile and scalable robot simulation framework*”, Intelligent Robot and Systems (IROS), pp. 1321-1326, 2013.
- [17] V. Martinez, “*Modeling of the flight dynamics of a quadrotor Helicopter*” [M.S. thesis], Cranfield University, Cranfield, UK, 2007.
- [18] A.Benini, A.Mancini and S.Longhi, “*An IMU/UWB/Vision-based Extended Kalman Filter for Mini-UAV Localization in Indoor Environment using 802.15.4a Wireless Sensor Network*”, Journal of Intelligent & Robotic Systems, vol. 40, no. 1-4, pp. 461-476, April 2013.
- [19] P.Villella, F.Rocchi, C.Avizzano and M.Bergamasco, “*Robot Relocalisation from Laser Scanning Constraints Using Minimalistic Environmental Models*”, in 7th IFAC Symposium on Intelligent Autonomous Vehicles, 2010.
- [20] V.Nguyen, A.Martinelli, N.Tomatis and R.Siegwart, “*A comparison of line extraction algorithms using 2D laser rangefinder for indoor mobile robotics*”, in Intelligent Robots and Systems, 2005.
- [21] R.Siegwart and O.Arras, “*Feature Extraction and Scene Interpretation for Map Based Navigation and Map Building*”, in LSA-CONF, Pittsburgh, 1997.
- [22] J.Leonard and H.Durrand-Whyte, “*Directed Sonar Sensing for Mobile Robot Navigation*”, 1992.
- [23] S.Thrun, W.Burgard et D.Fox, “*Probabilistic Robotics*”, The MIT press, 2005.
- [24] S. Grzonka, “*Mapping, State Estimation, and Navigation for Quadrotors and Human-Worn Sensor Systems*”, PhD Thesis, September 2011.
- [25] S.Grzonka, G.Grisetti and W.Burgard, “*A Fully Autonomous Indoor Quadrotor*”, IEEE Transactions on Robotics, vol. 28, no. 1, February 2012.
- [26] O.Khatib, “*Real-time obstacle avoidance for manipulators and mobile robots*”, 1985 IEEE International Conference on Robotics and Automation. Proceedings , vol. 2, pp. 500-505, 1985.
- [27] C.Connolly and R.A.Grupen, “*On the applications of harmonic functions to robotics*”, Journal of Robotic Systems, no. 10, pp. 931-946, 1993.
- [28] J.Borenstein and Y.Koren, “*Histogramic in-motion mapping for mobile robot avoidance*”, IEEE Journal of Robotics and Automation, vol. 7, no. 4, pp. 535-539, 1991.
- [29] R.Shade and P.Newman, “*Choosing where to go: Complete 3D exploration with stereo*”, in IEEE International Conference on Robotics and Automation, Shanghai, 2011.
- [30] R.A.Gupta, A.A.Massoud and M.Y.Chow, “*A Delay-Tolerant Potential-Field-Based Network Implementation of an Integrated Navigation System*”, IEEE Transactions on Industrial Electronics, vol. 57, no. 2, pp. 769-783, 2010.
- [31] H.Bay, T.Tuytelaars and L.V.Gool, “*SURF: Speed Up Robust Features*”, [Online]. Available: <http://www.vision.ee.ethz.ch/~surf/eccv06.pdf>
- [32] D.G.Lowe, “*Object recognition from local scale-invariant features*”, in Proceedings of the International Conference on Computer Vision, 1999.